CS559: BUILDING BLOCKS

Dr. Gagan Raj Gupta



REQUIREMENTS GATHERING

- Example of a twitter like service. Clarify requirements by asking questions like:
  - Will users of our service be able to post tweets and follow other people?
  - Should we also design to create and display the user's timeline?
  - Will tweets contain photos and videos?
  - Will we suppress disturbing content?
  - Are we focusing on the backend only, or are we developing the front-end too?
  - Will users be able to search tweets?
  - Do we need to display hot trending topics?
  - Will there be any push notification for new (or important) tweets?
  - Will we validate and suppress fake/spam/misleading tweets?



BACK-OF-THE-ENVELOPE ESTIMATE

- It is always a good idea to estimate the scale of the system we're going to design. This will also help later when we focus on scaling, partitioning, load balancing, and caching.
  - What scale is expected from the system (e.g., number of new tweets, number of tweet views, number of timeline generations per sec., etc.)?
  - How much storage will we need? We will have different storage requirements if users can have photos and videos in their tweets.
  - What network bandwidth usage are we expecting? This will be crucial in deciding how we will manage traffic and balance load between servers.



SYSTEM INTERFACE DEFINITION

- Define what APIs are expected from the system. This will establish the exact contract expected from the system and ensure if we haven't gotten any requirements wrong.
- Some examples of APIs for our Twitter-like service will be:
  - postTweet(user_id, timestamp, tweet_data, tweet_location, user_location, tags, …)
  - generateTimeline(user_id, current_time, user_location, …)
  - markTweetFavorite(user_id, tweet_id, timestamp, …)

BQ 2.1: Write APIs for other features in the previous slide

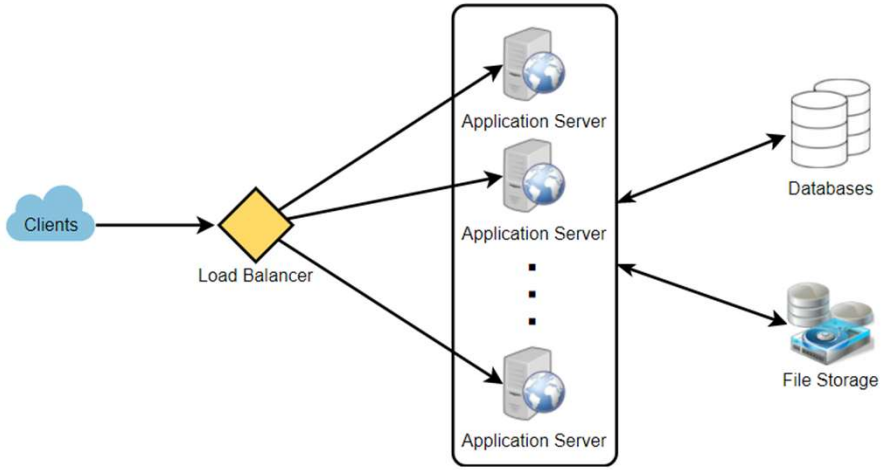


DEFINING DATA MODEL

- Identify various system entities, how they will interact with each other, and different aspects of data management like storage, transportation, encryption, etc.
- Here are some entities for a Twitter-like service:
  - **User:** UserID, Name, Email, DoB, CreationDate, LastLogin, etc.
    Tweet: TweetID, Content, TweetLocation, NumberOfLikes, TimeStamp, etc.
    UserFollow: UserID1, UserID2
    FavoriteTweets: UserID, TweetID, TimeStamp
- Which database system should we use? Will NoSQL like Cassandra best fit our needs, or should we use a MySQL-like solution? What kind of block storage should we use to store photos and videos?



HIGH LEVEL DESIGN



For Twitter, at a high level, we will need:

- Multiple application servers to serve all the read/write requests
- Load balancers in front of them for traffic distributions.
- If we're assuming that we will have a lot more read traffic (compared to write), we can decide to have separate servers to handle these scenarios.
- On the back-end, we need an efficient database that can store all the tweets and support a large number of reads.
- We will also need a distributed file storage system for storing photos and videos.

DETAILED DESIGN

Exploring options

- Since we will be storing a massive amount of data, how should we partition our data to distribute it to multiple databases? Should we try to store all the data of a user on the same database? What issue could it cause?
- How will we handle hot users who tweet a lot or follow lots of people?
- Since users' timeline will contain the most recent (and relevant) tweets, should we try to store our data so that it is optimized for scanning the latest tweets?
- How much and at which layer should we introduce cache to speed things up?
- What components need better load balancing?



IDENTIFYING BOTTLENECKS

- Is there any single point of failure in our system? What are we doing to mitigate it?
- Do we have enough replicas of the data so that we can still serve our users if we lose a few servers?
- Similarly, do we have enough copies of different services running such that a few failures will not cause a total system shutdown?
- How are we monitoring the performance of our service? Do we get alerts whenever critical components fail or their performance degrades?



BUILDING BLOCKS (IN AWS)

- Typically, computer systems need three things:
  - Compute
  - Storage
  - Network
- AWS provides the following
  - EC2: Get a complete computer in cloud in a few seconds
  - S3: Highly durable hash-table in the cloud. (**Key**: any string, **Value**: Data Blob up to 5 TB)
  - Lambda: Simplest way to run code in the cloud (Code runner)
  - Load Balancers: Manage load on the application servers
  - Route 53: DNS service
  - CloudFormation: Manage your infrastructure on AWS



LAMBDA

- If EC2 is a complete computer in the cloud, Lambda is a code runner in the cloud
- With EC2 you get an OS, file system, access to the server's hardware, etc.
- With Lambda, you just upload some code and Amazon runs it for you
- Strong points of Lambda:
  - It's the simplest way to run code in the cloud.
  - It abstracts away everything except for a function interface, which you get to fill in with the code you want to run.
- Lambda is not a general-purpose application host!
- Use cases (examples):
  - Perform operations on S3 data: E.g. Re-size Images uploaded to S3
  - Filter Data being loaded to services such as Kinesis
  - Enhance Application Load Balancers, CloudFront,



LOAD BALANCERS

- ELB comes in three variants:
  - Classic: Legacy option for very old AWS accounts
  - ALBs: Proper reverse proxies that sit between your application and the internet
  - NLB: Works by routing packets from clients to servers (transparently)
- Application Load Balancers (ALB)
  - Every request to your application gets handled by the load balancer first
  - LB makes another request to your application and finally forwards the response from your application to the caller.
  - Features: support sophisticated routing rules, redirects, responses from Lambda functions, authentication, sticky sessions, and many other things
- Both ALBs and NLBs support TLS/HTTPS, using for example AWS Certificate Manager.
  - AWS becomes the man-in-the-middle is able to see all your data! (decrypts and encrypts)
  - Alternative is to use TCP passthrough and manage certificates on applications hosts, renewals etc.



ROUTE 53

- Route 53 is a DNS service. It lets you translate domain names to IP addresses.
- There's nothing particularly special about Route 53's DNS capabilities.
  - First of all, it's good enough.
  - Secondly, it integrates very well with ELB.
- Few minor limitations such as the lack of support for ALIAS records (unless they point to AWS resources)



CLOUD FORMATION

- Lets you create and update the things you have in AWS without having to click around on the console or write fragile scripts.
- Even for development, the ability to tear down everything cleanly and recreate your AWS set up in one click is extremely valuable.
- With CloudFormation;
  - Define AWS resources as a YAML script (or JSON, YAML is much easier to read and modify).
  - Point CloudFormation to your AWS account, and it creates all the resources you defined.
  - If you run the script again without making any changes, CloudFormation won't do anything (it's idempotent).
  - If you make a change to one resource, it will change only that resource, plus any other resources that depend on the modified one (if necessary).
  - If you change your mind about an update, you can safely tell CloudFormation to roll it back.
  - You can also tell CloudFormation to tear down everything it created, and it will give you your AWS account back in the original state (with a few exceptions).
  - CONS: Be careful with manual changes



RULE OF THUMB

Virtual private cloud

Managed by CloudFormation	Managed Elsewhere
VPC configurations	DynamoDB tables
Security groups	Kinesis streams
Load balancers	Autoscale settings
Deployment pipelines	S3 buckets
IAM roles	

- Rule of thumb: Let CloudFormation manage AWS resources that are either static or change very rarely
- You may want to handle some of these things directly from your application, or you could have another simple script that sets them up separately.



URL SHORTENING SERVICE

- Benefits: space savings, less errors, user monitoring, ad services
- Functional Requirements
  - Size of the short-url
  - Redirection to the original url
  - Link expiry
- Non-Functional Requirements
  - High-availability
  - Minimal latency
  - Short-urls must not be predictable (??)
- Extended Requirments
  - Analytics
  - Access via API



CAPACITY ESTIMATION

- Read: Write ratio = 100:1
- Assume we have 500 M new URL shortening per month
  - Number of re-directions:
- Queries per second:
- URL redirections per second:
- Storage estimate for 5 years:
- Bandwidth estimates
- Memory estimate: 20% cache



CAPACITY ESTIMATES: URL SHORTENING

Types of URLs	Time estimates
New URLs	200/s
URL redirections	20K/s
Incoming data	100KB/s
Outgoing data	10 MB/s
Storage for 5 years	15 TB
Memory for cache	170 GB

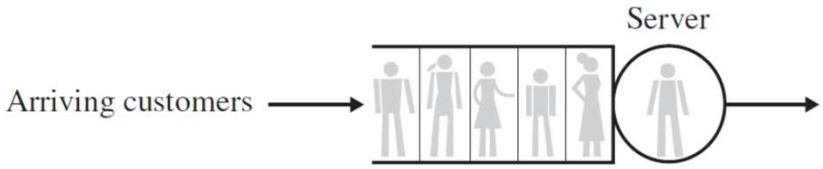


YELP OR NEARBY SERVICE

- Let's design a Yelp like service, where users can search for nearby places like restaurants, theaters, or shopping malls, etc., and can also add/view reviews of places.
- Similar Services: Proximity server.



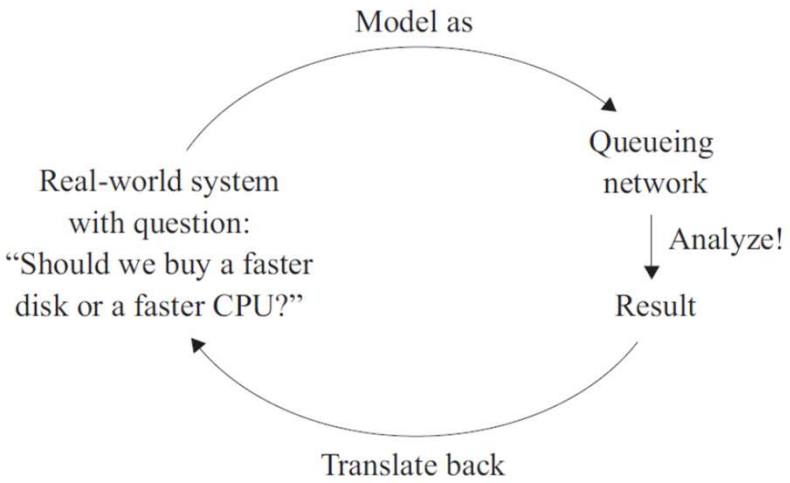
QUEUING THEORY 101



- Bank: Waiting in line at the Bank/ATM, wondering why there are not more tellers/machine.
- Super-market: Why is the express lane is for 8 items or less rather than 15 items or less, or whether it might be best to actually have *two* express lanes, one for 8 items or less and the other for 15 items or less.
- CPU uses a time-sharing scheduler to serve a queue of jobs waiting for CPU time.
- Computer disk serves a queue of jobs waiting to read or write blocks.
- Router in a network serves a queue of packets waiting to be routed. The router queue is a finite capacity queue, in which packets are dropped when demand exceeds the buffer space.
- Memory banks serve queues of threads requesting memory blocks.
- Databases sometimes have lock queues, where transactions wait to acquire the lock on a record.
- Server farms consist of many servers, each with its own queue of jobs.



ANALYTICAL THINKING



GOALS OF A QUEUING THOERIST

- *Predicting* the system performance: Predicting mean delay or delay variability or the probability that delay exceeds some Service Level Agreement (SLA). However, it can also mean predicting the number of jobs that will be queueing or the mean number of servers being utilized (e.g., total power needs), or any other such metric.
- Finding a superior system *design* to improve performance. Commonly this takes the form of capacity planning, where one determines which additional resources to buy to meet delay goals (e.g., is it better to buy a faster disk or a faster CPU, or to add a second slow disk). Many times, however, without buying any additional resources at all, one can improve performance just by deploying a smarter scheduling policy or different routing policy to reduce delays.
- Given the importance of smart scheduling in computer systems, all of Week 10 of this course is devoted to understanding scheduling policies.



ASSUMPTIONS

- Construct a stochastic model that represents the jobs or customers in a system and its service dynamics,
- These stochastic models are not always analytically tractable with respect to solving for performance.
- *Markovian assumptions*, such as assuming Exponentially distributed service demands or a Poisson arrival process, greatly simplify the analysis
- The arrival process of book orders on Amazon might be reasonably well approximated by a Poisson process, given that there are many independent users, each independently submitting requests at a low rate (although this all breaks down when a new Harry Potter book comes out).
- However, in some cases Markovian assumptions are very far from reality; for example, in the case in which service demands of jobs are highly variable or are correlated.
- There are often ways to work around these assumptions, many of which are shown in this book, such as using phase-type distributions and matrix-analytic methods,

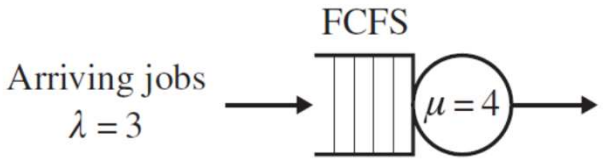


KEY METRICS

- **response time** is the time from when a job arrives until it completes service, a.k.a. sojourn time.



SINGLE-SERVER NETWORK



- **Service Order:** Order in which jobs will be served by the server, e.g. First-Come-First-Served (FCFS).
- **Average Arrival Rate:** This is the average rate, λ, at which jobs arrive to the server (e.g., $\lambda = 3$ jobs/sec).
- **Mean Interarrival Time:** This is the average time between successive job arrivals (e.g., $1/\lambda = 1/3$ sec).
- **Service Requirement, Size:** The "size" of a job is typically denoted by the random variable $\mathcal{S}$. This is the time it would take the job to run on this server if there were no other jobs around (no queueing). In a queueing model, the size (a.k.a. service requirement) is typically associated with the server (e.g., this job will take 5 seconds on this server).
- **Mean Service Time:** This is the expected value of $\mathcal{S}$, namely the average time required to service a job on this CPU, where "service" does not include queueing time. In Figure 2.2, $\mathbf{E}[\mathcal{S}] = 1/4$ sec.
- **Average Service Rate** This is the average rate, μ, at which jobs are served (e.g., $\mu = 4$ jobs/sec $= 1/\mathbf{E}[\mathcal{S}]$).

TRANSLATING TO QUEUEING SYSTEM

- The average arrival rate of jobs is 3 jobs per second.
- Jobs have different service requirements,
  - but the average number of cycles required by a job is 5,000 cycles per job.
  - The CPU speed is 20,000 cycles per second.
- That is, an average of 15,000 cycles of work arrive at the CPU each second, and the CPU can process 20,000 cycles of work a second.
- In the queueing-theoretic way of talking, we would never mention the word "cycle." Instead, we would simply say
  - The average arrival rate of jobs is 3 jobs per second.
  - The average rate at which the CPU can service jobs is 4 jobs per second.

